

データ分析ゼミ：環境構築

はじめての開発環境と Markdown

授業用教材・試作版

目次

この教材について	5
到達目標	5
授業での使い方	5
1 環境構築	7
1.1 作業前の準備	7
1.2 macOS: Homebrew · VS Code · Git	7
1.3 Windows: winget · VS Code · Git	8
1.4 演習: Markdown を書いてプレビューする	9
1.5 GitHub アカウントを準備する	10
1.6 Git の初期設定	11
1.7 GitHub Education を申請する	11
1.8 完了チェックと提出	12
1.9 困ったとき	12

この教材について

プログラミングを始めるための道具をそろえ、VS Code で最初の学習メモを作成します。対象は、データ分析をこれから学ぶ大学生です。

到達目標

- 自分の OS に合った方法で VS Code と Git を導入できる。
- Markdown ファイルを編集し、整形されたプレビューを表示できる。
- GitHub アカウントを準備し、GitHub Education を申請できる。

授業での使い方

所要時間は 75 ～ 105 分 が目安です。ダウンロード時間と Education の審査期間により前後します。授業全 15 回・サブゼミ全 10 回のうち、初回の環境構築に使い、申請のフォローをサブゼミで行う想定です。

作業	目安
導入・OS 確認	5 分
VS Code・Git の導入	25 ～ 40 分
Markdown 演習	15 分
GitHub・Git 設定	15 ～ 20 分
Education 申請・完了確認	15 ～ 25 分

自分の OS の節だけを実施し、その後は共通の演習へ進みます。既に導入済みのソフトは、バージョンを確認できれば先へ進めます。

この章では Python・Jupyter・Quarto のインストールは扱いません。Markdown 演習は VS Code の標準機能だけで行えます。

試作版：2026 年 9 月 15 日確認。画面や申請条件が変わった場合は、各節の公式資料を参照してください。Web 版と PDF 版は同じ Quarto 原稿から生成しています。

1. 環境構築

1.1 作業前の準備

自分が管理できる PC、安定したインターネット接続、受信できる個人メールと大学メール、現在の在学状況を確認できる学生証などを用意します。大学管理の PC では、ソフトの導入可否を担当者に確認してください。

1.1.1 道具の役割

道具	何に使うか
Homebrew / winget	アプリをコマンドで導入・更新するパッケージマネージャ
VS Code	文章やプログラムを書くエディタ
Git	ファイルの変更履歴を記録するソフト
GitHub	Git で管理した資料を共有する Web サービス
Markdown	見出しや箇条書きを簡単な記号で書く形式

「コマンド」は PC への文字による指示です。Mac ではターミナル、Windows では PowerShell に入力して Enter を押します。コード枠の中だけをコピーし、複数行は上から 1 行ずつ実行します。行頭に表示されるプロンプト（\$ や PS C:\...>）は入力しません。

パスワード・認証コード・回復コードは、教材のメモや提出物に書かないでください。

1.2 macOS: Homebrew・VS Code・Git

Windows の人は [Windows の手順](#)へ進みます。

1.2.1 OS とターミナルを確認する

1. Apple メニューの「この Mac について」を開き、macOS のバージョンとチップを確認します。「Apple M...」なら Apple Silicon、「Intel」なら Intel Mac です。
2. [Homebrew の動作要件](#)を確認します。対応外の OS の場合は担当者へ相談します。
3. Spotlight（Command+Space）で「ターミナル」を検索し、起動します。

1.2.2 Homebrew を導入する

まず次を実行します。バージョンが表示されれば、インストール作業は省略できます。

```
brew --version
```

未導入なら [Homebrew 公式サイト](#)のインストールコマンドを確認して実行します。以下は確認時点のコマンドです。折り返されていても 1 行です。

```
/bin/bash -c "$(curl -fsSL https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

これは公式のインストール用スクリプトを取得し、実行する指示です。画面に出る変更内容を読んで進めます。パスワードを求められたら Mac へのログインパスワードを入力します。入力中に

文字や丸印が表示されなくても正常です。Command Line Tools の導入が始まった場合は完了を待ちます。

完了画面の **Next steps** にコマンドが表示されたら、その指示を実行します。これは Homebrew を探す場所 (PATH) をターミナルに知らせる設定です。通常の導入先は Apple Silicon が `/opt/homebrew`、Intel が `/usr/local` ですが、自分の画面に表示された指示を使ってください。

ターミナルを開き直し、確認します。

```
brew --version
brew doctor
```

前者でバージョン、後者で診断結果が表示されれば実行できています。警告がある場合は内容を確認します。警告すべてが授業を止める問題とは限りません。判断できない場合はエラー文を担当者に見せます。

1.2.3 VS Code と Git を導入する

```
brew install --cask visual-studio-code
brew install git
```

完了後、アプリケーションから Visual Studio Code を起動します。ターミナルを開き直して確認します。

```
code --version
git --version
```

両方でバージョンが出れば完了です。code が見つからない場合は、VS Code で Command+Shift+P を押し、Shell Command: Install 'code' command in PATH を検索して実行し、ターミナルを開き直します。

公式資料: [VS Code の Homebrew パッケージ](#)、[VS Code の macOS 設定](#)。

1.3 Windows: winget · VS Code · Git

Mac の人は [Markdown 演習](#) へ進みます。

1.3.1 PowerShell と winget を確認する

1. 設定の「システム」→「バージョン情報」で Windows のバージョンを確認します。大学の方針に従い、サポートされている OS を使います。
2. スタートメニューで「PowerShell」を検索して、通常の権限で起動します。Windows Terminal を使う場合は PowerShell のタブを選びます。
3. 次のコマンドを実行します。

```
winget --version
```

v で始まるバージョンが出れば利用できます。見つからなければ Microsoft Store で Microsoft 提供の「アプリ インストーラー (App Installer)」をインストールまたは更新し、PowerShell を開き直します。利用できない場合は [winget 公式資料](#) を確認します。

1.3.2 VS Code と Git を導入する

次のコマンドを 1 行ずつ実行します。

```
winget install --id Microsoft.VisualStudioCode -e --source winget
winget install --id Git.Git -e --source winget
```

--id はアプリの識別名、-e は完全一致、--source winget は検索する配布元の指定です。利用条件やインストール確認が出たら内容を読んで進みます。管理者の許可を求められた場合は、自分が開始したインストールか確認してください。管理された PC では担当者に依頼します。

インストール終了後、PowerShell を閉じて開き直します。

```
code --version
git --version
```

両方でバージョンが出れば完了です。スタートメニューから Visual Studio Code を起動できることも確認します。

公式資料: [winget install](#)、[VS Code の Windows 設定](#)。

1.4 演習: Markdown を書いてプレビューする

目安は 15 分です。完成時には、左に Markdown の原稿、右に整形された表示が並びます。

1.4.1 作業フォルダとファイルを作る

Finder またはエクスプローラーで自分のホームフォルダ（ユーザーフォルダ）を開き、seminar-work というフォルダを作成します。同名のフォルダがあれば、その中を確認して使います。

VS Code の「ファイル」→「フォルダーを開く」から、そのフォルダを選びます。自分で作ったフォルダに対する信頼確認が出たら、内容を確認して信頼します。

コマンドでも開けます。フォルダ作成後、Mac のターミナルでは次を実行します。

```
cd ~/seminar-work
code .
```

Windows の PowerShell では次を実行します。

```
cd "$env:USERPROFILE\seminar-work"
code .
```

. は「今いるフォルダ」です。VS Code の左側のエクスプローラーから「新しいファイル」を選び、setup-check.md と名付けます。

1.4.2 Markdown を入力する

次の内容を入力またはコピーします。コード枠の外側のバッククォートは入力しません。

```
# 環境構築チェック

## 使用環境

- OS: macOS または Windows
- エディタ: VS Code
- バージョン確認: `git --version`

## 完了した作業

- [ ] VS Code を起動した
- [ ] Git のバージョンを確認した
- [ ] GitHub アカウントを作成した

## 参考資料

- [GitHub Docs](https://docs.github.com/)
```

Mac は Command+S、Windows は Ctrl+S で保存します。ファイル名が setup-check.md.txt になっていないことを確認してください。

記号	表示の意味
#、##	見出し、ひとつ下の階層の見出し
-	箇条書き
- [], - [x]	未完了、完了のチェック項目
バッククォートで囲む	コマンドなどを本文と区別して表示
[表示名](URL)	リンク

や - の後には半角スペースを入れます。段落やリストの前後には空行を入れます。

1.4.3 通常プレビューを開く

原稿のタブを選び、次のショートカットを押します。

操作	macOS	Windows
プレビュー	Command+Shift+V	Ctrl+Shift+V
横にプレビュー	Command+K → V	Ctrl+K → V

横に表示するときは、原稿へ戻ってから操作します。矢印は「最初のキーの組み合わせを押して離し、続けて V を押す」という意味です。

メニューから開く場合は、コマンドパレット (Mac: Command+Shift+P、Windows: Ctrl+Shift+P) で `Markdown: Open Preview to the Side` を検索します。

1.4.4 編集が反映されるか確認する

1. 左の原稿で OS 名を自分の環境に合わせて書き換えます。
2. VS Code 起動の項目を - [x] `VS Code` を起動した に変更します。
3. ## 今日わかったこと を追加し、その下に学んだことを 1 行書きます。
4. 右のプレビューで見出し・チェック状態・文章が更新されたか確認します。
5. 保存して、ファイルを開き直しても変更が残ることを確認します。

演習の成功条件: 原稿とプレビューを左右に表示でき、編集結果が反映され、保存したファイルが残っていることです。チェック項目は原稿側で書き換えます。GitHub の項目は後の作業が終わってから更新します。

この演習では VS Code 標準の Markdown プレビューを使います。追加の拡張機能は不要です。プレビューのセキュリティ設定は既定の Strict のままで表示できます。

公式資料: [Markdown 編集とプレビュー](#)。

1.5 GitHub アカウントを準備する

1.5.1 アカウントとメール認証

既に個人アカウントがある人は、それを利用します。

1. [GitHub](#) を開き、Sign up から個人アカウントを作成します。
2. 継続して使えるユーザー名を選びます。ユーザー名は公開プロフィールや URL に使われます。
3. 画面の案内に従い、登録メールに届く確認を完了します。
4. サインイン後、Settings → Emails で大学メールを追加します。
5. 大学メール宛での確認を完了し、認証済みになっていることを確認します。

卒業後も使う個人メールを登録しておく、大学メールが失効した後も連絡先を維持できます。公開プロフィールにメールを表示するかどうかは別に設定できます。

公式資料: [アカウントの作成](#)。

1.5.2 二要素認証を設定する

二要素認証（2FA）は、パスワードに加えて認証アプリなどで本人確認を行う仕組みです。

1. Settings → Password and authentication を開きます。
2. Two-factor authentication の設定を開始し、認証アプリなどの方法を画面に従って登録します。
3. 回復コードを保存します。スマートフォンを紛失しても使えるよう、パスワード管理アプリなど安全な場所に保管します。
4. 設定画面で二要素認証が有効であることを確認します。

回復コードはアカウントへのアクセスを回復するための秘密情報です。GitHub のリポジトリ、`setup-check.md`、授業の提出物には保存しません。

公式資料: [二要素認証の設定](#)。

1.6 Git の初期設定

Git は変更履歴に作成者の名前とメールを記録します。この設定は GitHub へのログインとは別です。公開する履歴にも含まれるため、設定する値を先に決めます。

GitHub の Settings → Emails でメール非公開の設定を確認すると、GitHub 提供の `noreply` アドレスを確認できます。個人のメールを公開したくない場合は、そのアドレスをそのままコピーして使います。文字列を推測して作らないでください。

次の引用符内を自分の値に置き換えてから、ターミナルまたは PowerShell で実行します。名前は履歴に表示したい名前、メールは GitHub で認証済みのメールか GitHub 提供の非公開用メールにします。

```
git config --global user.name "自分の表示名"
git config --global user.email "自分のメールアドレス"
git config --global init.defaultBranch main
```

`--global` はこの PC の自分のユーザー全体に適用する指定です。既に設定済みなら、変更前に下の確認コマンドで現在の値を見てください。main は新しく履歴管理を始めるときの既定ブランチ名です。

```
git config --global --get user.name
git config --global --get user.email
git config --global --get init.defaultBranch
```

指定した名前・メール・main が順に表示されれば完了です。全設定を自分で確認するには `git config --global --list` を使えますが、出力全体を提出する必要はありません。接続設定、コミット、push は後の授業で扱います。

公式資料: [Git の初期設定](#)、[コミットのメール設定](#)。

1.7 GitHub Education を申請する

1.7.1 条件と書類を確認する

申請には個人の GitHub アカウントが必要です。学生向けの主な条件は 13 歳以上で、学位・卒業資格につながる教育課程に現在在籍していることです。学校発行の認証可能なメール、または現在の在籍を証明する資料を用意します。学校によって大学メールの認証が必要です。

在学資料には、氏名・学校名・現在の在籍期間が確認できる学生証、履修時間割、成績証明書、在学証明書などを使います。日付のない学生証しかない場合は、現在の在籍が分かる追加資料を準備します。

1.7.2 申請を送信する

1. GitHub へサインインします。
2. [Education benefits](#) を開き、申請開始（Start an application）を選びます。

3. 学生として申請し、学校名・大学メールなど、フォームが求める情報を入力します。
4. 在学証明を求められたら、文字と日付が読める資料をフォームの案内に沿って提出します。
5. 内容を確認して送信し、申請状況を確認します。

ここでの完了は申請送信までです。審査待ちは環境構築の失敗ではありません。`setup-check.md`に「申請済み・審査待ち」などの状態だけ記録します。証明書の画像は学習メモや公開リポジトリに入れません。

大学メールが発行されない学校では、申請画面の案内に沿って在学資料や学校のメール発行方針を示します。ドメインが認識されない場合は公式ヘルプの案内からサポートへ連絡します。

条件・申請方法の一次情報: [GitHub Education への学生申請](#)。

1.7.3 承認後の確認

承認されたら Education benefits で利用できる特典を確認します。GitHub Copilot などの特典は別途有効化が必要な場合があり、反映に数日かかることもあります。有料プランしか表示されない場合は、そのまま購入せず、承認状態と公式案内を確認します。

公式資料: [学生向け Copilot の設定](#)。

1.8 完了チェックと提出

次の項目を自分で確認し、`setup-check.md`を更新します。

- ☐ `brew --version` または `winget --version` が表示される。
- ☐ VS Code を起動でき、`code --version` が表示される。
- ☐ `git --version` が表示される。
- ☐ `setup-check.md` を作成・保存できる。
- ☐ Markdown の通常プレビューと左右分割プレビューを開ける。
- ☐ 原稿の編集がプレビューに反映される。
- ☐ Git の名前・メール・既定ブランチを確認した。
- ☐ GitHub にサインインでき、登録メールと大学メールを認証した。
- ☐ 二要素認証を有効にし、回復コードを安全に保管した。
- ☐ Education を申請した。または、未申請の理由を整理した。

授業での提出例は `setup-check.md` です。OS、確認したソフトのバージョン、完了項目、分かったこと、困っていることを記録します。アカウント名・メール・認証情報・学生証は含めません。提出先は授業担当者が指定します。

確認問題：Git と GitHub はそれぞれ何をする道具ですか。Markdown の原稿とプレビューはどのような関係ですか。自分の言葉で説明してみましょう。

1.9 困ったとき

1.9.1 Mac で brew が見つからない

導入後の PATH 設定が未反映の可能性があります。インストール最後の Next steps を確認し、指示を実行してターミナルを開き直します。[Homebrew 導入資料](#)を参照してください。

1.9.2 code または git が見つからない

インストール後も古いターミナルを使っている可能性があります。VS Code 内のターミナルも含めて開き直します。Mac の `code` は [macOS の節](#)の PATH 設定を確認します。Windows はインストール完了を確認し、[公式設定手順](#)を参照します。

1.9.3 Windows で winget が見つからない

App Installer が未導入・未更新の可能性があります。Microsoft Store で確認します。Store が利用制限されている場合は、大学の管理担当者へ相談します。[winget 公式資料](#)を参照してください。

1.9.4 Markdown プレビューが開かない・更新されない

ファイルの拡張子が `.md` か確認します。原稿のタブを選び、コマンドパレットから Markdown のプレビューを開きます。別ファイルのプレビューを見ていないか確認し、保存します。横に開くキーは同時押しを 2 段階に分けます。[VS Code 公式資料](#)も確認してください。

1.9.5 GitHub の確認メールが届かない

迷惑メール、メールアドレスの入力間違い、大学メールの受信制限を確認します。Settings → Emails で再送できます。[メール認証の公式案内](#)を参照してください。

1.9.6 Education で学校が見つからない・証明が受理されない

大学メールの認証、学校名、書類の氏名・日付・可読性を確認します。却下理由が表示されたらそれを読み、必要な資料をそろえて再申請します。解決しない場合は[学生申請の公式案内](#)からサポートへ進みます。

担当者へ相談するときは、OS、実行した操作、エラー文、試した対処を伝えます。スクリーンショットに秘密情報が写っていないか確認してから共有します。